

Pipeline Workflow

Detecting AI-Generated Text Using ML and NLP Techniques

GitHub: <https://github.com/SabirShah1402/Detecting-AI-Generated-Text-Project>

4.1 Data pre-processing and feature engineering

The pipeline is fully modular. Each stage runs as an independently executable notebook or script.

Stage	Steps	Output Artefact
1. Ingestion	Load HC3 via load_dataset("Hello-SimpleAI/HC3", "all"). Load Kaggle CSV. Validate schemas and null counts.	Raw DataFrames
2. Standardise	Flatten HC3 human_answers and chatgpt_answers lists into individual rows. Rename to unified schema: text, label (0/1), source, dataset.	Unified DataFrame
3. Clean	Remove duplicate texts, drop rows < 10 words, strip whitespace and URLs, enforce str dtype throughout.	Clean DataFrame
4. Feature Eng.	Compute per-row: word_count, char_count, avg_sentence_length, punctuation_density, type-token ratio.	DataFrame + features
5. Balance	Compute minority/majority label ratio. Apply random undersampling of majority if ratio < 0.70.	Balanced DataFrame
6. Split	Stratified 70/15/15 split on label. Save train.csv, val.csv, test.csv. Verify no text overlap between splits.	3 CSV files
7. TF-IDF	Fit TfidfVectorizer(max_features=50000, ngram_range=(1,2)) on train only. Transform all three splits.	Sparse matrices
8. Embeddings	For CNN: GloVe-100d embeddings (max_len=200). For RoBERTa: roberta tokenizer (max_length=256, padding=True).	Tokenised tensors
9. Training	Train all three models. Early stopping (patience=3, monitor val F1) for CNN and RoBERTa.	Model checkpoints
10. HP Tuning	GridSearchCV for LR. Manual grid for CNN. TrainingArguments sweep for RoBERTa. Log all to results_log.csv.	Tuned models
11. Evaluation	Accuracy, Macro F1, ROC-AUC, Confusion Matrix on test set. Cross-dataset F1 test (HC3 ↔ Kaggle).	Metrics report
12. Deployment	Export best model. Streamlit app: text input → preprocess → inference → display result + confidence.	Live web app

4.2 Model Architectures

Model	Architecture	Hyperparameters Tuned
Model 1 LR Baseline	TF-IDF (unigram+bigram, 50k features) → Logistic Regression. L2	C ∈ {0.001, 0.01, 0.1, 1, 10, 100}; penalty ∈ {l1, l2}; class_weight; max_iter

Model	Architecture	Hyperparameters Tuned
	penalty, liblinear/saga solver. Probability calibration for ROC-AUC.	
Model 2 Hybrid CNN	Stream A: stylometric features (word_count, punct_density, avg_sent_len) → Dense(64, ReLU). Stream B: GloVe-100d → Conv1D(filters, kernel) → GlobalMaxPool → Dense(64, ReLU). Concat → Dropout → Dense(1, sigmoid).	filters ∈ {64,128,256}; kernel ∈ {3,5}; dropout ∈ {0.3,0.5}; lr ∈ {1e-3,5e-4,1e-4}; batch_size ∈ {32,64}
Model 3 RoBERTa	roberta-base (125M params, HuggingFace). SequenceClassification head added. Fine-tuned with AdamW, linear warmup schedule. HuggingFace Trainer API, early stopping on val F1 (patience=3).	lr ∈ {2e-5,3e-5,5e-5}; warmup_ratio ∈ {0.06,0.10}; weight_decay; batch_size ∈ {16,32}; max_length

- **Training environments:** Stages 1–6 and Model 1: CPU (scikit-learn). Model 2 (Hybrid CNN): Google Colab GPU (PyTorch). Model 3 (RoBERTa): Colab A100/T4 via HuggingFace Trainer.
- **Dummy baseline check:** A majority-class dummy classifier defines the absolute performance floor. Every trained model must strictly outperform it — failure raises a GitHub Issue for investigation before proceeding.
- **Experiment tracking:** All training runs (timestamp, hyperparams, val F1, epochs) logged to results_log.csv and committed to GitHub.

4.3 Hyperparameter Optimisation

- **Logistic Regression:** scikit-learn GridSearchCV, 5-fold stratified CV. Grid: $C \in \{0.001, 0.01, 0.1, 1, 10, 100\} \times \text{penalty} \in \{l1, l2\}$. Scoring: macro F1. Best params saved to model card and results_log.csv.
- **Hybrid CNN:** Manual grid over $lr \in \{1e-3, 5e-4, 1e-4\}$, filters ∈ {64, 128, 256}, kernel_size ∈ {3, 5}, dropout ∈ {0.3, 0.5}. Each config trained to max 20 epochs with early stopping. Results in cnn_tuning.csv.
- **RoBERTa:** HuggingFace TrainingArguments sweep: $lr \in \{2e-5, 3e-5, 5e-5\}$, warmup_ratio ∈ {0.06, 0.10}, batch_size ∈ {16, 32}. Best checkpoint selected by val F1. Full training logs via Trainer callback.
- **Reporting:** All tuning results (before/after comparison) compiled in the report appendix table, showing gains from default to tuned for each model.

4.4 Evaluation Metrics

Metric	Rationale	Visualisation
Macro F1-Score	Primary KPI. Balances precision and recall equally across both classes; robust to class imbalance.	Bar chart — all 3 models
Overall Accuracy	Intuitive correctness measure. Secondary to F1 in imbalanced conditions.	Comparison table
ROC-AUC	Threshold-independent ranking quality. Captures full model performance curve irrespective of cutoff.	Overlaid ROC curves
Precision / Recall	Disentangles false-positive vs. false-negative trade-offs separately per class.	Per-class table

Metric	Rationale	Visualisation
Confusion Matrix	Exposes systematic failure modes — e.g., AI text consistently misclassified as human.	Heatmap per model
Cross-dataset F1	Train on HC3 → test on Kaggle (and reverse). Measures generalisation beyond training distribution.	Generalisation table

All three models are compared in a single unified table in the report. Visualisations: (a) overlaid ROC curve plot, (b) per-model confusion matrix heatmaps, (c) macro F1 bar chart comparison.

4.5 Application Deployment

- **Framework:** Streamlit (Python). `app.py` loads the best model checkpoint on startup in CPU mode — no GPU required at inference.
- **User flow:** Text input → Submit → output panel shows: (a) label (Human / AI-Generated), (b) confidence bar, (c) top-5 TF-IDF feature contributions, (d) stylometric summary (word count, avg sentence length).
- **Input validation:** Minimum 10 words enforced. Empty or too-short inputs return descriptive warnings rather than errors.
- **Deployment:** Local: `streamlit run app.py`. Cloud option: Streamlit Community Cloud via GitHub repo integration. Model checkpoint in `/models/`, loaded once at startup with `joblib.load()` or `torch.load()`.